


```

NotStrictlyPositiveException ||
    relativeAccuracy <= 0                && ex = NotStrictlyPositiveException ||
    absoluteAccuracy <= 0                && ex = NotStrictlyPositiveException
||
    minImalIterationCount <= 0           && ex = NotStrictlyPositiveException
||
    minImalIterationCount >= maximalIterationCount
                                                && ex =

NumberIsTooSmallException ||
isNaN=1                && ex = NotANumberException ||
    isInfinite=1        && ex = NotFiniteNumberException ||

    min >= max                && ex =

NumberIsTooLargeException ||
evaluations_used > maxEval && ex = TooManyEvaluationsException ||
evaluations_used <= maxEval && ex = MaxCountExceededException
||

evaluations_used <= minEval

                                                && ex = None

where
    valid_inputs ==
        f ≠ null and
        maxEval > 0 and
        (relativeAccuracy > 0 ∨ absoluteAccuracy > 0) and
        minImalIterationCount > 0 and
        minImalIterationCount < maximalIterationCount and
        (¬isNaN(min)) and (¬isNaN(max)) and
        (¬isInfinite(min)) and (¬isInfinite(max)) and
        min < max
end_process

end_module

```

- Java Code

```

/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
 * 2.0

```

```

* (the "License"); you may not use this file except in compliance with
* the License. You may obtain a copy of the License at
*
*     http://www.apache.org/licenses/LICENSE-2.0
*
* Unless required by applicable law or agreed to in writing, software
* distributed under the License is distributed on an "AS IS" BASIS,
* WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
* See the License for the specific language governing permissions and
* limitations under the License.
*/
package org.apache.commons.math4.legacy.analysis.integration;
import org.apache.commons.numbers.core.ArithmeticUtils;
import org.apache.commons.math4.legacy.exception.NumberIsTooLargeExcept
ion;
import org.apache.commons.math4.core.jdkmath.JdkMath;
import org.apache.commons.math4.legacy.analysis.UnivariateFunction;
import org.apache.commons.math4.legacy.exception.NullArgumentException;
import org.apache.commons.math4.legacy.exception.NotStrictlyPositiveExc
eption;
import org.apache.commons.math4.legacy.exception.NumberIsTooSmallExcept
ion;
import org.apache.commons.math4.legacy.exception.NumberIsTooLargeExcept
ion;
import org.apache.commons.math4.legacy.exception.NotANumberException;
import org.apache.commons.math4.legacy.exception.NotFiniteNumberExcepti
on;
import org.apache.commons.math4.legacy.exception.TooManyEvaluationsExce
ption;
import org.apache.commons.math4.legacy.exception.MaxCountExceededExcept
ion;
/**
 * Implements the
 * <a href="https://en.wikipedia.org/wiki/Riemann_sum#Midpoint_rule"> M
idpoint
 * Rule</a> for integration of real univariate functions. For reference
, see
 * <b>Numerical Mathematics</b>, ISBN 0387989595, chapter 9.2.
 * <p>
 * The function should be integrable.
 * </p>
 *
 * @since 3.3

```

```

*/
public class MidPointIntegrator extends BaseAbstractUnivariateIntegrator {
    /**
     * Maximum number of iterations for midpoint. 39 = floor(log3(263)),
     the
     * maximum number of triplings allowed before exceeding 64-bit bounds.
     */
    private static final int MIDPOINT_MAX_ITERATIONS_COUNT = 39;
    /**
     * Build a midpoint integrator with given accuracies and iterations counts.
     *
     * @param relativeAccuracy    relative accuracy of the result
     * @param absoluteAccuracy    absolute accuracy of the result
     * @param minimalIterationCount minimum number of iterations
     * @param maximalIterationCount maximum number of iterations
     * @exception org.apache.commons.math4.legacy.exception.NotStrictlyPositiveException if
     *
     *         minimal
     *
     *         number
     *
     *         of
     *
     *         iterations
     *
     *         is
     *
     *         not
     *
     *         strictly
     *
     *         positive
     * @exception org.apache.commons.math4.legacy.exception.NumberIsTooSmallException if
     *
     *         maximal
     *
     *         number
     *
     *         of

```

```
*
    iterations
*
    is
*
    lesser
*
    than
*
    or
*
    equal
*
    to
*
    the
*
    minimal
*
    number
*
    of
*
    iterations
* @exception NumberIsTooLargeException
    if
*
    maximal
*
    number
*
    of
*
    iterations
*
    is
*
    greater
*
    than
*
    39.
*/
```

```

    public MidPointIntegrator(final double relativeAccuracy, final double
absoluteAccuracy,
        final int minimalIterationCount, final int maximalIterationCount) {
        super(relativeAccuracy, absoluteAccuracy, minimalIterationCount, maxi
malIterationCount);
        if (maximalIterationCount > MIDPOINT_MAX_ITERATIONS_COUNT) {
            throw new NumberIsTooLargeException(maximalIterationCount, MIDPOINT_
MAX_ITERATIONS_COUNT, false);
        }
    }
    /**
     * Build a midpoint integrator with given iteration counts.
     *
     * @param minimalIterationCount minimum number of iterations
     * @param maximalIterationCount maximum number of iterations
     * @exception org.apache.commons.math4.legacy.exception.NotStrictlyPos
itiveException if
     *
     *         minimal
     *
     *         number
     *
     *         of
     *
     *         iterations
     *
     *         is
     *
     *         not
     *
     *         strictly
     *
     *         positive
     * @exception org.apache.commons.math4.legacy.exception.NumberIsTooSma
llException if
     *
     *         maximal
     *
     *         number
     *
     *         of
     *
     *         iterations

```

```
*
    is
*
    lesser
*
    than
*
    or
*
    equal
*
    to
*
    the
*
    minimal
*
    number
*
    of
*
    iterations
* @exception NumberIsTooLargeException
    if
*
    maximal
*
    number
*
    of
*
    iterations
*
    is
*
    greater
*
    than
*
    39.
*/
public MidPointIntegrator(final int minimalIterationCount, final int m
aximalIterationCount) {
    super(minimalIterationCount, maximalIterationCount);
```

```

    if (maximalIterationCount > MIDPOINT_MAX_ITERATIONS_COUNT) {
        throw new NumberIsTooLargeException(maximalIterationCount, MIDPOINT_
MAX_ITERATIONS_COUNT, false);
    }
}
/**
 * Construct a midpoint integrator with default settings. (max iterati
on count
 * set to {@link #MIDPOINT_MAX_ITERATIONS_COUNT})
 */
public MidPointIntegrator() {
    super(DEFAULT_MIN_ITERATIONS_COUNT, MIDPOINT_MAX_ITERATIONS_COUNT);
}
/**
 * Compute the n-
th stage integral of midpoint rule. This function should only
 * be called by API integrate() in the package. To save t
ime it
 * does not verify arguments - caller does.
 * <p>
 * The interval is divided equally into  $3^n$  sections rather than an ar
bitrary m
 * sections because this configuration can best utilize the already co
mputed
 * values.
 * </p>
 *
 * @param n the stage of 1/3 refinement. Must be large
r than
 * 0.
 * @param previousStageResult Result from the previous call to the {@c
ode stage}
 * method.
 * @param min Lower bound of the integration interval.
 * @param diffMaxMin Difference between the lower bound and up
per bound
 * of the integration interval.
 * @return the value of n-th stage integral
 * @throws org.apache.commons.math4.legacy.exception.TooManyEvaluation
sException if
 *
 * the
 *
 * maximal

```

```

*
    number
*
    of
*
    evaluations
*
    is
*
    exceeded.
*/
private double stage(final int n, double previousStageResult, double min, double diffMaxMin) {
    // number of points in the previous stage. This stage will contribute
    //  $2 \cdot 3^{n-1}$  more points.
    final long np = ArithmeticUtils.pow(3L, n - 1);
    double sum = 0;
    // spacing between adjacent new points
    final double spacing = diffMaxMin / np;
    final double leftOffset = spacing / 6;
    final double rightOffset = 5 * leftOffset;
    double x = min;
    for (long i = 0; i < np; i++) {
        // The first and second new points are located at the new midpoints
        // generated when each previous integration slice is split into 3.
        //
        // |-----x-----|
        // |--x--|--x--|--x--|
        sum += computeObjectiveValue(x + leftOffset);
        sum += computeObjectiveValue(x + rightOffset);
        x += spacing;
    }
    // add the new sum to previously calculated result
    return (previousStageResult + sum * spacing) / 3.0;
}
/** {@inheritDoc} */
@Override
protected double doIntegrate() {
    final double min = getMin();
    final double diff = getMax() - min;
    final double midPoint = min + 0.5 * diff;
    double oldt = diff * computeObjectiveValue(midPoint);
    while (true) {
        iterations.increment();
    }
}

```

```

    final int i = iterations.getCount();
    final double t = stage(i, oldt, min, diff);
    if (i >= getMinimalIterationCount()) {
        final double delta = JdkMath.abs(t - oldt);
        final double rLimit = getRelativeAccuracy() * (JdkMath.abs(oldt) +
JdkMath.abs(t)) * 0.5;
        if (delta <= rLimit || delta <= getAbsoluteAccuracy()) {
            return t;
        }
    }
    oldt = t;
}
}
}
}
class MPIVT {
    // ----- helpers -----
    private static void headline(String name) {
        System.out.println("\n==== " + name + " ====");
    }
    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        }
    }
    // ----- sample functions -----
    static final UnivariateFunction F_ZERO = x -> 0.0;
    static final UnivariateFunction F_CONST = x -> 5.0; //  $\int_a^b 5 \, dx = 5(b-a)$ 
    static final UnivariateFunction F_LIN = x -> x - 2.0; //  $\int_0^4 (x-2) \, dx = 0$ 
    static final UnivariateFunction F_SIN = Math::sin; //  $\int_0^\pi \sin = 2$ 
    public void Vtest() {
        final double REL = 1e-10;
        final double ABS = 1e-10;
        final int MIN_ITER = 3;
    }
}

```

```

final int MAX_ITER = 12;
final int BIG_BUDGET = 1 << Math.min(28, (MAX_ITER + 3)); // 防止移位
超界
MidPointIntegrator M = new MidPointIntegrator(REL, ABS, MIN_ITER, MAX_ITER);
// VT1: f == null -> NullArgumentException
expectThrow("VT1 f=null", () -> M.integrate(1000, null, 0.0, 1.0), NullArgumentException.class);
// VT2: maxEval <= 0 -> NotStrictlyPositiveException
expectThrow("VT2 maxEval=0", () -> M.integrate(0, F_SIN, 0.0, 1.0), NotStrictlyPositiveException.class);
expectThrow("VT2b maxEval<0", () -> M.integrate(-5, F_SIN, 0.0, 1.0), NotStrictlyPositiveException.class);
// VT3: relativeAccuracy <= 0 (ctor) -> NotStrictlyPositiveException
expectThrow("VT3 relAcc<=0 (ctor)", () -> new MidPointIntegrator(0.0, 1e-8, 3, 10), NotStrictlyPositiveException.class);
// VT4: absoluteAccuracy <= 0 (ctor) -> NotStrictlyPositiveException
expectThrow("VT4 absAcc<=0 (ctor)", () -> new MidPointIntegrator(1e-8, 0.0, 3, 10), NotStrictlyPositiveException.class);
// VT5: minimalIterationCount <= 0 (ctor) -> NotStrictlyPositiveException
expectThrow("VT5 minIter<=0 (ctor)", () -> new MidPointIntegrator(REL, ABS, 0, 10), NotStrictlyPositiveException.class);
// VT6: minimalIterationCount >= maximalIterationCount (strict) -> NumberIsTooSmallException
expectThrow("VT6 minIter>=maxIter (ctor)", () -> new MidPointIntegrator(REL, ABS, 5, 5), NumberIsTooSmallException.class);
// VT7: NaN endpoints -> NotANumberException
expectThrow("VT7 NaN at min", () -> M.integrate(1000, F_LIN, Double.NaN, 1.0), NotANumberException.class);
expectThrow("VT7b NaN at max", () -> M.integrate(1000, F_LIN, 0.0, Double.NaN), NotANumberException.class);
// VT8: Infinite endpoints -> NotFiniteNumberException
expectThrow("VT8 -Infinity at min", () -> M.integrate(1000, F_SIN, Double.NEGATIVE_INFINITY, 1.0), NotFiniteNumberException.class);
expectThrow("VT8b +Infinity at max", () -> M.integrate(1000, F_SIN, 0.0, Double.POSITIVE_INFINITY), NotFiniteNumberException.class);

```

```

// VT9:
expectThrow("VT9 min==max", () -> M.integrate(1000, F_CONST, 3.14, 3.
14), NumberIsTooLargeException.class);
expectThrow("VT9b min>max", () -> M.integrate(1000, F_CONST, 1.0, -
1.0), NumberIsTooLargeException.class);
// VT10:
expectThrow("VT10 budget exhausted (maxEval=1)", () -> M.integrate(1,
F_SIN, 0.0, 10.0),
    TooManyEvaluationsException.class);
// VT11:
try {
    MidPointIntegrator Mtight = new MidPointIntegrator(1e-30, 1e-
30, 1, 2);
    expectThrow("VT11 hit max iterations (tight accuracy, small band)",
        () -> Mtight.integrate(BIG_BUDGET, F_SIN, 0.0, 100.0), MaxCountExc
eededException.class);
} catch (Throwable t) {
    System.out.println("VT11 note: " + t.getClass().getSimpleName() + "
- " + t.getMessage());
}
// VT12:
headline("VT12 ZERO small interval within budget");
double r12 = M.integrate(BIG_BUDGET, F_ZERO, -1e-6, 1e-6);
System.out.printf("VT12 result=%.17g%n", r12);
// VT13
headline("VT13 CONST small interval");
double a13 = 2.0, b13 = 2.0000002;
double r13 = M.integrate(BIG_BUDGET, F_CONST, a13, b13);
System.out.printf("VT13 result=%.17g (expected=%g)%n", r13, 5.0 * (b1
3 - a13));
// VT14:
expectThrow("VT14a SIN [0,π] hits max iterations (REL=ABS=1e-
10, MAX_ITER=12)", () -> {
    MidPointIntegrator Mtight = new MidPointIntegrator(REL, ABS, MIN_ITE
R, MAX_ITER);
    int budget14a = Math.max(BIG_BUDGET, 1_000_000);
    Mtight.integrate(budget14a, F_SIN, 0.0, Math.PI);
}, MaxCountExceededException.class);
// VT15:
headline("VT15 LIN [0,4]");
double r15 = M.integrate(Math.max(BIG_BUDGET, 500_000), F_LIN, 0.0, 4
.0);
System.out.printf("VT15 result=%.17g |err|=%g%n", r15, Math.abs(r15 -
0.0));

```

```

// VT16:
headline("VT16 near-degenerate but valid");
double a16 = 3.0;
double eps16 = Math.max(1e-9, 16 * Math.ulp(a16)); // 16xULP + 下限
double r16 = M.integrate(BIG_BUDGET, F_CONST, a16, a16 + eps16);
System.out.printf("VT16 result=%.17g (expected=%g, eps=%g)%n", r16, 5
.0 * eps16, eps16);
// VT17:
headline("VT17 wide finite SIN sanity [-100,100]");
double r17 = M.integrate(Math.max(BIG_BUDGET, 2_000_000), F_SIN, -
100.0, 100.0);
System.out.printf("VT17 result=%.17g%n", r17);
// VT18:
headline("VT18 tight accuracy on tiny interval");
MidPointIntegrator Mtiny = new MidPointIntegrator(1e-14, 1e-
14, 3, 12);
double r18 = Mtiny.integrate(BIG_BUDGET, F_SIN, 0.0, 1e-6);
System.out.printf("VT18 result=%.17g%n", r18);
// VT19:
headline("VT19 small iter band but feasible");
MidPointIntegrator Mband = new MidPointIntegrator(1e-8, 1e-8, 2, 4);
double r19 = Mband.integrate(1 << 18, F_CONST, -1e-4, 1e-4);
System.out.printf("VT19 result=%.17g (expected=%g)%n", r19, 5.0 * (2e
-4));
// VT20:
headline("VT20 medium interval LIN");
double r20 = M.integrate(Math.max(BIG_BUDGET, 400_000), F_LIN, -
5.0, 7.25);
System.out.printf("VT20 result=%.17g%n", r20);
System.out.println("\n[Done] MidPointIntegrator V-
method cases executed.");
}
}
//ft
class MPIFT {
// ----- helpers -----
private static void headline(String name) {
System.out.println("\n==== " + name + " ====");
}
private static void expectThrow(String name, Runnable r, Class<?> expe
cted) {
System.out.println("---- " + name + " ----");
try {
r.run();

```

```

        System.out.printf("[FAIL] %s should throw %s%n", name, expected.getSimpleName());
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
        System.out.printf("[%s] %s threw expected%n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
    }
}

// ----- sample functions -----
static final UnivariateFunction F_ZERO = x -> 0.0;
static final UnivariateFunction F_CONST = x -> 5.0; //  $\int_a^b 5 \, dx = 5(b-a)$ 
static final UnivariateFunction F_LIN = x -> x - 2.0; //  $\int_0^4 (x-2) \, dx = 0$ 
static final UnivariateFunction F_SIN = Math::sin; //  $\int_0^\pi \sin = 2$ 
public void Ftest() {
    final double REL = 1e-10;
    final double ABS = 1e-10;
    final int MIN_ITER = 3;
    final int MAX_ITER = 12;
    final int BIG_BUDGET = 1 << Math.min(28, (MAX_ITER + 3)); // 防止移位超界
    MidPointIntegrator M = new MidPointIntegrator(REL, ABS, MIN_ITER, MAX_ITER);
    // ----- Fuzz pack: FT01 - FT20 (MidPointIntegrator)
    // -----
    int FZ_BUDGET = Math.max(BIG_BUDGET, 1 << Math.min(28, (MAX_ITER + 5))); // 充足预算
    // FT01: reversed interval (extreme) -> NumberIsTooLargeException
    expectThrow("FT01 reversed interval (extreme)",
        () -> M.integrate(1000, F_SIN, 5.005341669616643E307, -1.1209145429141500E308),
        NumberIsTooLargeException.class);
    // FT02: degenerate interval (min==max at extreme) -> NumberIsTooLargeException
    expectThrow("FT02 degenerate interval (min==max, extreme)",
        () -> M.integrate(1000, F_CONST, -1.7976931348623157E308, -1.7976931348623157E308),
        NumberIsTooLargeException.class);
    // FT03: NaN min -> NotANumberException
    expectThrow("FT03 NaN min", () -> M.integrate(1000, F_LIN, Double.NaN, 1.0), NotANumberException.class);
    // FT04: NaN max -> NotANumberException

```

```

    expectThrow("FT04 NaN max", () -> M.integrate(1000, F_SIN, 0.0, Double.NaN), NotANumberException.class);
    // FT05: +Infinity at max -> NotFiniteNumberException
    expectThrow("FT05 +Infinity at max", () -> M.integrate(1000, F_SIN, -10.0, Double.POSITIVE_INFINITY),
        NotFiniteNumberException.class);
    // FT06: -Infinity at min -> NotFiniteNumberException
    expectThrow("FT06 -Infinity at min", () -> M.integrate(1000, F_CONST, Double.NEGATIVE_INFINITY, 10.0),
        NotFiniteNumberException.class);
    // FT07: budget=1 on wide interval -> TooManyEvaluationsException
    expectThrow("FT07 budget=1 on wide interval", () -> M.integrate(1, F_SIN, -1.0e3, 1.0e3),
        TooManyEvaluationsException.class);
    // FT08: budget=1 on  $[0, \pi]$  -> TooManyEvaluationsException
    expectThrow("FT08 budget=1 on  $[0, \pi]$ ", () -> M.integrate(1, F_SIN, 0.0, Math.PI),
        TooManyEvaluationsException.class);
    // FT09: ZERO on tiny symmetric interval (success)
    headline("FT09 ZERO tiny symmetric interval");
    double mpf09 = M.integrate(FZ_BUDGET, F_ZERO, -1e-12, 1e-12);
    System.out.printf("FT09 result=%.17g%n", mpf09);
    // FT10: CONST near-degenerate safe epsilon (success)
    headline("FT10 CONST near-degenerate safe");
    double a10 = 3.141592653589793;
    double eps10 = Math.max(1e-9, 16 * Math.ulp(a10));
    double mpf10 = M.integrate(FZ_BUDGET, F_CONST, a10, a10 + eps10);
    System.out.printf("FT10 result=%.17g (expected=%g, eps=%g)%n", mpf10, 5.0 * eps10, eps10);
    // FT11: SIN on moderate interval (success)
    headline("FT11 SIN moderate interval");
    double mpf11 = M.integrate(FZ_BUDGET, F_ZERO, -1e-12, 1e-12);
    System.out.printf("FT11 result=%.17g%n", mpf11);
    // FT12: LIN on wide finite interval (success)
    headline("FT12 LIN wide finite");
    double mpf12 = M.integrate(FZ_BUDGET, F_LIN, -2.5e2, 4.0e2);
    System.out.printf("FT12 result=%.17g%n", mpf12);
    // FT13: CONST on extreme-wide finite with tiny budget ->
    // TooManyEvaluationsException
    expectThrow("FT13 CONST extreme-wide, tiny budget",
        () -> M.integrate(1, F_CONST, -5.000000000000000E307, 5.000000000000000E307),
        TooManyEvaluationsException.class);

```

```

// FT14: ZERO on very wide finite (success, integral=0)
headline("FT14 ZERO very wide finite");
double mpf14 = M.integrate(FZ_BUDGET, F_ZERO, -
7.654321098765432E307, 7.654321098765432E307);
System.out.printf("FT14 result=%.17g%n", mpf14);
// FT15: SIN on huge-offset small window (success)
headline("FT15 SIN huge-offset small window");
double A15 = 1.2345678901234567E307;
double mpf15 = M.integrate(FZ_BUDGET, F_SIN, A15, A15 + 1.0);
System.out.printf("FT15 result=%.17g%n", mpf15);
// FT16: CONST tiny negative interval (success)
headline("FT16 CONST tiny negative interval");
double mpf16 = M.integrate(FZ_BUDGET, F_CONST, -1.000000000000000E-
12, -9.999999999999990E-13);
System.out.printf("FT16 result=%.17g (expected≈%g)%n", mpf16, 5.0 * (
1.0e-14));
// FT17: ctor relAcc≤0 fuzz -> NotStrictlyPositiveException
expectThrow("FT17 ctor relAcc≤0", () -> new MidPointIntegrator(0.0,
1e-8, 3, 10),
NotStrictlyPositiveException.class);
// FT18: integrate maxEval≤0 fuzz -> NotStrictlyPositiveException
expectThrow("FT18 maxEval≤0", () -> M.integrate(0, F_LIN, -
1.0, 1.0), NotStrictlyPositiveException.class);
// FT19: tight accuracy + small iter band (likely max-iter hit) ->
// MaxCountExceededException
try {
    MidPointIntegrator MtightFZ = new MidPointIntegrator(1e-30, 1e-
30, 1, 2);
    expectThrow("FT19 hit max iterations (tight acc, small band)",
    () -> MtightFZ.integrate(FZ_BUDGET, F_SIN, 0.0, 100.0), MaxCountEx
ceededException.class);
} catch (Throwable t) {
    System.out.println("FT19 note: " + t.getClass().getSimpleName() + "
- " + t.getMessage());
}
// FT20: ZERO around subnormal range (success)
headline("FT20 ZERO around subnormal range");
double mpf20 = M.integrate(FZ_BUDGET, F_ZERO, -1.0e-300, 1.0e-300);
System.out.printf("FT20 result=%.17g%n", mpf20);
System.out.println("\n[Done] MidPointIntegrator Fuzzing cases execute
d.");
}
}
//Z3

```

```

class MPIZ3 {
    // ---- helpers ----
    private static void headline(String name) {
        System.out.println("\n==== " + name + " ===");
    }
    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            System.out.printf("[FAIL] %s should throw %s\n", name, expected.getSimpleName());
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            System.out.printf("[%s] %s threw expected\n", t.getClass().equals(expected) ? "PASS" : "FAIL", name);
        }
    }
    // ---- functions ----
    static final UnivariateFunction F_ZERO = x -> 0.0;
    static final UnivariateFunction F_CONST = x -> 5.0;
    static final UnivariateFunction F_SIN = Math::sin;
    static final UnivariateFunction F_LIN = x -> x - 2.0;
    public void Z3test() {
        java.util.function.IntUnaryOperator roomyBudget = maxIter -> 1 << Math.min(28, maxIter + 10);
        // Z301 -----
        --
        /*
         * 01) f = null → ex = NullPointerException relativeAccuracy=0,
         * absoluteAccuracy=0, minimalIterationCount=0, maximalIterationCount=0,
         * maxEval=0, f_is_null=True ... ex = NullPointerException (code=1)
         */
        MidPointIntegrator M301 = new MidPointIntegrator(1e-3, 1e-3, 1, 2);
        expectThrow("Z301 f=null", () -> M301.integrate(10, null, 0.0, 1.0),
            NullPointerException.class);
        // Z302 -----
        --
        /*
         * 02) maxEval <= 0 → ex = NotStrictlyPositiveException rel=1/2, abs=1/2,
         * minIter=1, maxIter=2, maxEval=0

```

```

    */
    MidPointIntegrator M302 = new MidPointIntegrator(0.5, 0.5, 1, 2);
    expectThrow("Z302 maxEval<=0", () -> M302.integrate(0, F_ZERO, 0.0, 1
.0), NotStrictlyPositiveException.class);
    // Z303 -----
--
    /*
    * 03) relativeAccuracy <= 0 → ex = NotStrictlyPositiveException rel=
0, abs=1/2,
    * minIter=1, maxIter=2
    */
    expectThrow("Z303 relAcc<=0 (ctor)", () -> new MidPointIntegrator(0.0
, 0.5, 1, 2),
    NotStrictlyPositiveException.class);
    // Z304 -----
--
    /*
    * 04) absoluteAccuracy <= 0 → ex = NotStrictlyPositiveException rel=
1/2, abs=0,
    * minIter=1, maxIter=2
    */
    expectThrow("Z304 absAcc<=0 (ctor)", () -> new MidPointIntegrator(0.5
, 0.0, 1, 2),
    NotStrictlyPositiveException.class);
    // Z305 -----
--
    /*
    * 05) minimalIterationCount <= 0 → ex = NotStrictlyPositiveException
    * minIter=-1, maxIter=0
    */
    expectThrow("Z305 minIter<=0 (ctor)", () -> new MidPointIntegrator(0.
5, 0.5, -1, 0),
    NotStrictlyPositiveException.class);
    // Z306 -----
--
    /*
    * 06) minimalIterationCount ≥ maximalIterationCount → ex =
    * NumberIsTooSmallException minIter=1, maxIter=0
    */
    expectThrow("Z306 minIter>=maxIter (ctor)", () -> new MidPointIntegra
tor(0.5, 0.5, 1, 0),
    NumberIsTooSmallException.class);
    // Z307 -----
--

```

```

/*
 * 07) isNaN(min) → ex = NotANumberException
 */
MidPointIntegrator M307 = new MidPointIntegrator(0.5, 0.5, 1, 2);
expectThrow("Z307 isNaN(min)", () -> M307.integrate(10, F_LIN, Double
.NaN, 1.0), NotANumberException.class);
// Z308 -----
--

/*
 * 08) isNaN(max) → ex = NotANumberException
 */
MidPointIntegrator M308 = new MidPointIntegrator(0.5, 0.5, 1, 2);
expectThrow("Z308 isNaN(max)", () -> M308.integrate(10, F_LIN, 0.0, D
ouble.NaN), NotANumberException.class);
// Z309 -----
--

/*
 * 09) isInfinite(min) → ex = NotFiniteNumberException
 */
MidPointIntegrator M309 = new MidPointIntegrator(0.5, 0.5, 1, 2);
expectThrow("Z309 -
Infinity(min)", () -> M309.integrate(10, F_SIN, Double.NEGATIVE_INFINITY,
1.0),
    NotFiniteNumberException.class);
// Z310 -----
--

/*
 * 10) isInfinite(max) → ex = NotFiniteNumberException
 */
MidPointIntegrator M310 = new MidPointIntegrator(0.5, 0.5, 1, 2);
expectThrow("Z310 +Infinity(max)", () -> M310.integrate(10, F_SIN, 0.
0, Double.POSITIVE_INFINITY),
    NotFiniteNumberException.class);
// Z311 -----
--

/*
 * 11) min ≥ max → ex = NumberIsTooLargeException
 */
MidPointIntegrator M311 = new MidPointIntegrator(0.5, 0.5, 1, 2);
expectThrow("Z311 min==max", () -> M311.integrate(10, F_CONST, 0.0, 0
.0), NumberIsTooLargeException.class);
// Z312 -----
--

/*

```

```

    * 12) valid & evaluations_used > maxEval → ex = TooManyEvaluationsException
    */
    MidPointIntegrator M312 = new MidPointIntegrator(0.5, 0.5, 1, 2);
    expectThrow("Z312 budget exhausted (maxEval=1)", () -> M312.integrate
(1, F_SIN, 0.0, 1.0),
    TooManyEvaluationsException.class);
    // Z313 -----
--
    /*
    * 13) valid & evals ≤ maxEval & iters ≥ maxIters & !converged → ex =
    * MaxCountExceededException
    */
    try {
        MidPointIntegrator M313 = new MidPointIntegrator(1e-30, 1e-
30, 1, 2);
        expectThrow("Z313 hit max iterations (tight acc, small band)",
            () -> M313.integrate(roomyBudget.applyAsInt(2), F_SIN, 0.0, 100.0)
        ,
            MaxCountExceededException.class);
    } catch (Throwable t) {
        System.out.println("Z313 note: " + t.getClass().getSimpleName() + "
- " + t.getMessage());
    }
    // Z314 -----
--
    /*
    * 14) valid & converged_by_interval_shrinkage → ex = None
    */
    headline("Z314 success (interval shrinkage surrogate: ZERO tiny)");
    MidPointIntegrator M314 = new MidPointIntegrator(0.5, 0.5, 1, 2);
    double z314 = M314.integrate(256, F_ZERO, -1e-6, 1e-6);
    System.out.printf("Z314 result=%.17g%n", z314);
    // Z315 -----
--
    /*
    * 15) valid & converged_by_value_accuracy → ex = None
    */
    headline("Z315 success (value accuracy surrogate)");
    MidPointIntegrator M315 = new MidPointIntegrator(0.5, 1e-12, 1, 2);
    double z315 = M315.integrate(512, F_SIN, 0.0, 1e-3);
    System.out.printf("Z315 result=%.17g%n", z315);
    // Z316 -----
--

```

```

/*
 * 16) valid & both convergence flags true → ex = None
 */
headline("Z316 success (both flags surrogate: ZERO tinier)");
MidPointIntegrator M316 = new MidPointIntegrator(0.5, 0.5, 1, 2);
double z316 = M316.integrate(256, F_ZERO, -1e-9, 1e-9);
System.out.printf("Z316 result=%.17g%n", z316);
// Z317 -----
--

/*
 * 17) valid & minimalIterationCount=1 boundary → ex = None
 */
headline("Z317 success (minIter=1 boundary)");
MidPointIntegrator M317 = new MidPointIntegrator(0.5, 0.5, 1, 2);
double z317 = M317.integrate(512, F_CONST, 0.0, 1e-4);
System.out.printf("Z317 result=%.17g (expected=%g)%n", z317, 5.0 * 1e
-4);
// Z318 -----
--

/*
 * 18) valid & evaluations_used == maxEval → ex = None
 */
headline("Z318 success (within budget; eq surrogate)");
MidPointIntegrator M318 = new MidPointIntegrator(0.5, 0.5, 1, 2);
double z318 = M318.integrate(512, F_ZERO, -1.0, 1.0);
System.out.printf("Z318 result=%.17g%n", z318);
// Z319 -----
--

/*
 * 19) valid & tight interval ( $\max - \min \leq 1e-6$ ) → ex = None
 */
headline("Z319 success (tight interval  $\leq 1e-6$ )");
MidPointIntegrator M319 = new MidPointIntegrator(0.5, 0.5, 1, 2);
double z319 = M319.integrate(512, F_CONST, 3.0, 3.0 + 1e-6);
System.out.printf("Z319 result=%.17g (expected=%g)%n", z319, 5.0 * 1e
-6);
// Z320 -----
--

/*
 * 20) valid generic success (iters < maxIters) → ex = None
 */
headline("Z320 generic success");
MidPointIntegrator M320 = new MidPointIntegrator(0.5, 0.5, 1, 2);
double z320 = M320.integrate(512, F_LIN, 0.0, 4.0);

```

```

        System.out.printf("Z320 result=%.17g%n", z320);
        System.out.println("\n[Done] Z301-Z320 executed.");
    }
}

class MPItest {
    public static void main(String[] args) {
        // VT
        MPIVT t1 = new MPIVT();
        t1.Vtest();
        // FT
        MPIFT t2 = new MPIFT();
        // t2.Ftest();
        // Z3
        MPIZ3 t3 = new MPIZ3();
        // t3.Z3test();
    }
}

```

- Coverage report

1. Test case number: 20
2. V-Method testing branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
MidPointIntegrator.java	61.1 %	11	7	18
> MPIFT	0.0 %	0	2	2
> MPIZ3	0.0 %	0	2	2
> MidPointIntegrator	75.0 %	9	3	12
MidPointIntegrator(double, double, double)	50.0 %	1	1	2
doIntegrate()	100.0 %	6	0	6
stage(int, double, double, double)	100.0 %	2	0	2
> MPIVT	100.0 %	2	0	2
> MPItest		0	0	0

3. V-Method testing instruction coverage: 81.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
MidPointIntegrator.java	35.2 %	650	1,194	1,844
> MPIZ3	2.1 %	12	557	569
> MPIFT	2.4 %	12	485	497
> MPIVT	80.0 %	471	118	589
> MidPointIntegrator	81.9 %	140	31	171
MidPointIntegrator(double, double, double)	52.6 %	10	9	19
doIntegrate()	100.0 %	68	0	68
stage(int, double, double, double)	100.0 %	62	0	62
> MPItest	83.3 %	15	3	18

4. Fuzz testing branch coverage: 66.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
MidPointIntegrator.java	55.6 %	10	8	18
> MPIVT	0.0 %	0	2	2
> MidPointIntegrator	66.7 %	8	4	12
MidPointIntegrator(double, double, double)	50.0 %	1	1	2
doIntegrate()	83.3 %	5	1	6
stage(int, double, double, double)	100.0 %	2	0	2
> MPIFT	100.0 %	2	0	2
> MPItest		0	0	0

5. Fuzz testing instruction coverage: 81.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ MidPointIntegrator.java	24.6 %	454	1,390	1,844
> MPIVT	2.0 %	12	577	589
> MPItest	44.4 %	8	10	18
> MPIFT	59.2 %	294	203	497
▼ MidPointIntegrator	81.9 %	140	31	171
MidPointIntegrator(double, double,	52.6 %	10	9	19
doIntegrate()	100.0 %	68	0	68
stage(int, double, double, double)	100.0 %	62	0	62

6. Z3 branch coverage: 66.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ MidPointIntegrator.java	55.6 %	10	8	18
> MPIFT	0.0 %	0	2	2
> MPIVT	0.0 %	0	2	2
▼ MidPointIntegrator	66.7 %	8	4	12
MidPointIntegrator(double, double,	50.0 %	1	1	2
doIntegrate()	83.3 %	5	1	6
stage(int, double, double, double)	100.0 %	2	0	2
> MPIZ3	100.0 %	2	0	2
> MPItest	0.0 %	0	0	0

7. Z3 instruction coverage: 81.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ MidPointIntegrator.java	35.2 %	650	1,194	1,844
> MPIVT	2.0 %	12	577	589
> MPIFT	2.4 %	12	485	497
▼ MidPointIntegrator	81.9 %	140	31	171
MidPointIntegrator(double, double,	52.6 %	10	9	19
doIntegrate()	100.0 %	68	0	68
stage(int, double, double, double)	100.0 %	62	0	62
> MPIZ3	82.8 %	471	98	569
> MPItest	83.3 %	15	3	18